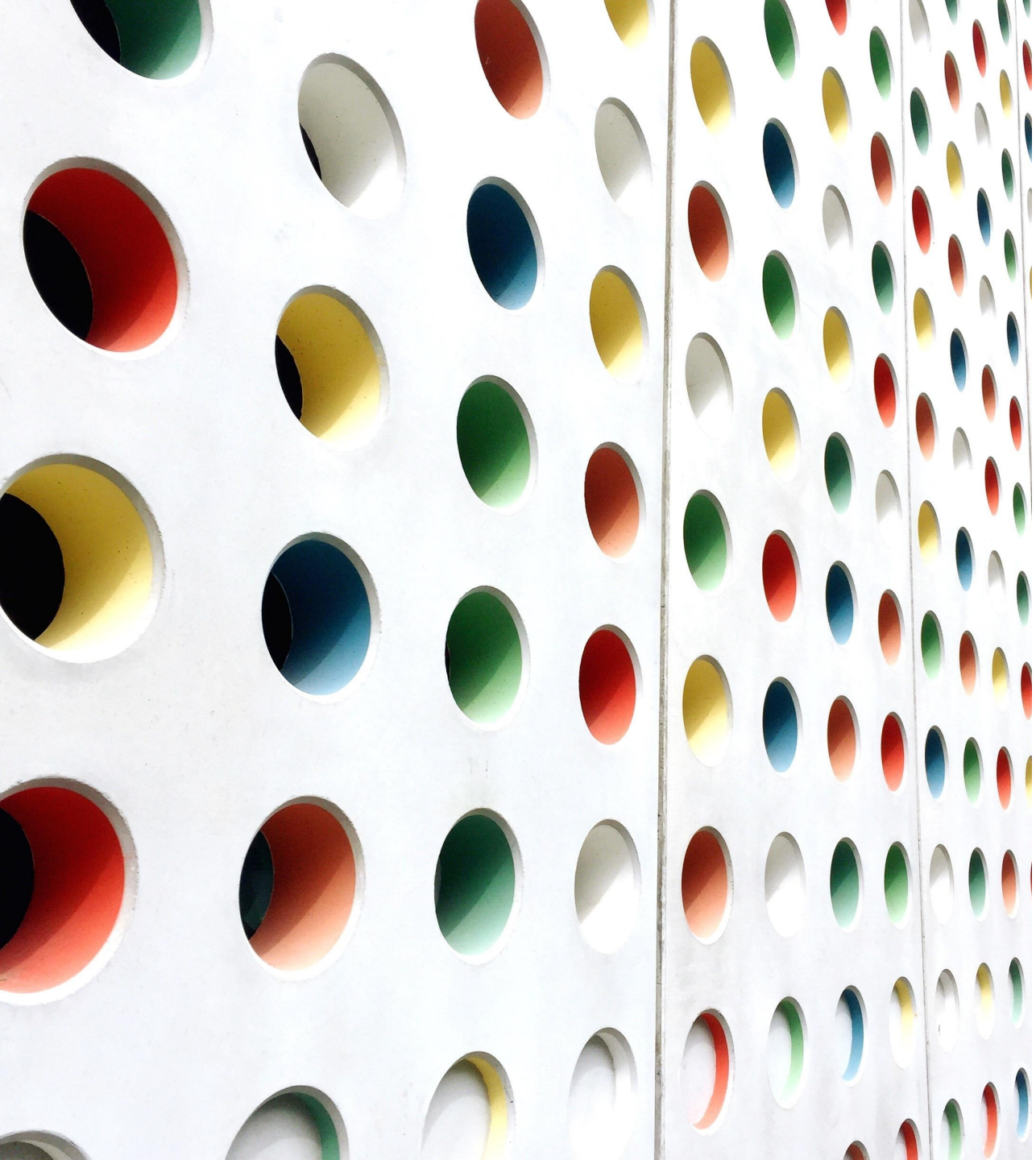


Semantic Annotation and Retrieval: RDF

BY

SANA RIZWAN

Complex Data Structure — RDF

Reification

- In order to group and order RDF triples, there are a couple of syntactical primitives defined to provide containers and collections in the form of lists.
- RDF offers a means to make statements about statements via reification.

1. Open Containers

Containers are open groups and contain unspecified numbers of resources or literals and possibly duplicates.

Three types of containers (semantics of the three types of containers is identical, and the different classes may be used informally only)

- `rdf:Bag` defines an unordered group of resources or literals.
- `rdf:Seq` defineds an ordered group of resources or literals.
- `rdf:Alt` represents a group of resources or literals that are alternatives; that is, only one of the values can be selected.

Open RDF containers

Resource is typed as rdf:Bag, rdf:Seq or rdf:Alt, and, syntactically, the members of the container are attached to it via special purpose numbered membership properties of the form rdf:_1, rdf:_2, etc. The RDF/XML syntax provides the property rdf:li to avoid the explicit numbering of members. Each occurrence of rdf:li is internally turned into a numbered property rdf:_1, rdf:_2, etc. to form the corresponding RDF graph

Open RDF containers

Turtle

```
<http://example.org/doc.html>
  a exs:Report ;
  dc:creator [ a rdf:Bag ;
               rdf:_1 <http://example.org#Fabien> ;
               rdf:_2 <http://example.org#York> ].
```

RDF/XML

```
<exs:Report rdf:about="http://example.org/doc.html">
  <dc:creator>
  <rdf:Bag>
    <rdf:li rdf:resource="http://example.org#Fabien"/>
    <rdf:li rdf:resource="http://example.org#York"/>
  </rdf:Bag>
</dc:creator>
</exs:Report>
```

Triple and Turtle example

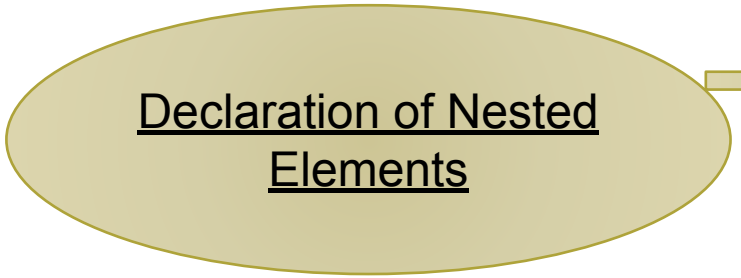
Triples

(http://example.org/doc.html , rdf:type , exs:Report)
(http://example.org/doc.html , dc:creator , exr:Fabien)
(http://example.org/doc.html , dc:creator , exr:York)
(http://example.org/doc.html , exs:theme , exr:Music)
(http://example.org/doc.html , exs:nbPages , "23")

```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.
@prefix xsd: <http://www.w3.org/2001/XMLSchema#>.
@prefix exs: <http://example.org/schema#>.
<http://example.org/doc.html> a exs:Report ;
                              exs:theme <http://example.org#Music>,
                                        <http://example.org#History> ;
                              exs:nbPages "23"^^xsd:int.
```

2. Closed Collections

- There is no way to restrict the number of members a container contains.
- Provide closed lists of resources or literals, possibly with duplicates, and contain, as per the RDF recommendation, only the specified members
- Lists are of type rdf:List with the property rdf:first pointing to the first member and a property rdf:rest recursively pointing to the list of the remaining members or to rdf:nil if the end of the list is reached, rdf:nil represents an empty list.
- Collections are declared as nested elements included via a property that has the attribute rdf:parseType = "Collection"



RDF/XML

```
<exs:Report rdf:about="http://example.org/doc.html">
  <exs:chapters rdf:parseType="Collection">
    <rdf:Description rdf:about="http://example.org/doc.html#chapter1"/>
    <rdf:Description rdf:about="http://example.org/doc.html#chapter2"/>
    <rdf:Description rdf:about="http://example.org/doc.html#chapter3"/>
  </exs:chapters>
</exs:Report>
```

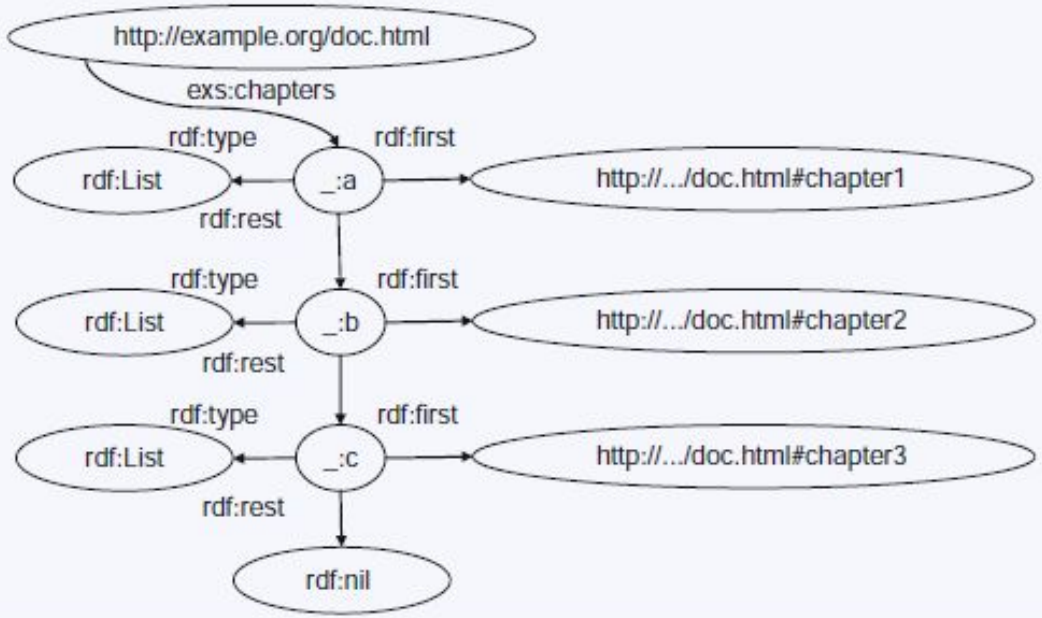
RDF serializations of closed lists

Turtle

```
<http://example.org/doc.html>
a exs:Report ;
xs:chapters (<http://example.org/doc.html#chapter1>
             <http://example.org/doc.html#chapter2>
             <http://example.org/doc.html#chapter3> ).
```

RDF/XML

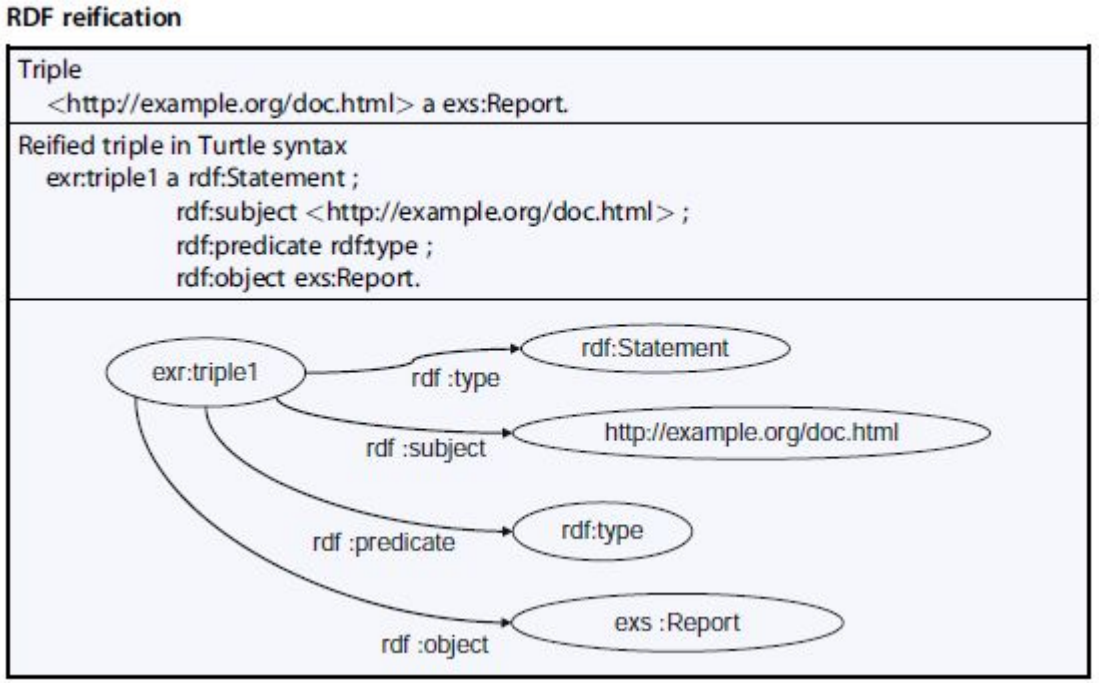
```
<exs:Report rdf:about="http://example.org/doc.html">
  <exs:chapters rdf:parseType="Collection">
    <rdf:Description rdf:about="http://example.org/doc.html#chapter1"/>
    <rdf:Description rdf:about="http://example.org/doc.html#chapter2"/>
    <rdf:Description rdf:about="http://example.org/doc.html#chapter3"/>
  </exs:chapters>
</exs:Report>
```



3. Semantic-Less Reification

Model yields a new resource to which additional information can be attached; for example, metadata about a given triple. The properties that RDF provides to explicitly represent and describe triples without asserting them.

- The use of RDF reification is rather discouraged as the semantics of reification are unclear and reified statements are quite problematic.
- RDF provides the constructs to write reifications, in RDF asserting the reification is not the same as asserting the original statement.
- Reification expands the initial triple into a total of five triples (a triple plus a reification quad) and the link between the initial triple and its reification is not maintained. Metadata about statements should preferably be attached to the data source instead of the reified triple.



RDFS

Define domain-specific classes and properties to describe these resources and organize them in hierarchies. These schemas are also published and exchanged in RDF.

RDF Schema (**Resource Description Framework Schema**, variously abbreviated as **RDFS**, **RDF(S)**, **RDF-S**, or **RDF/S**) is a set of classes with certain properties using the RDF extensible knowledge representation data model, providing basic elements for the description of ontologies, otherwise called RDF vocabularies, intended to structure RDF resources. These resources can be saved in a triplestore to reach them with the query language SPARQL.

RDFS Constructs – RDFS Classes

RDFS constructs are the RDFS classes, associated properties and utility properties built on the limited vocabulary of RDF.

Classes

- **rdfs:Resource** is the class of everything. All things described by RDF are resources.
- **rdfs:Class** declares a resource as a class for other resources.

An example of an rdfs:Class is foaf:Person in the Friend of a Friend (FOAF) vocabulary. An instance of foaf:Person is a resource that is linked to the class foaf:Person using the rdf:type property

Following formal expression of the natural-language sentence : 'John is a Person'.

ex:John rdf:type foaf:Person

The definition of rdfs:Class is recursive: rdfs:Class is the class of classes, and so it is an instance of itself.

rdfs:Class rdf:type rdfs:Class

The other classes described by the RDF and RDFS specifications are:

- **rdfs:Literal** – literal values such as strings and integers. Property values such as textual strings are examples of RDF literals. Literals may be plain or typed.
- **rdfs:Datatype** – the class of datatypes. rdfs:Datatype is both an instance of and a subclass of rdfs:Class. Each instance of rdfs:Datatype is a subclass of rdfs:Literal.
- **rdf:XMLLiteral** – the class of XML literal values. rdf:XMLLiteral is an instance of rdfs:Datatype (and thus a subclass of rdfs:Literal).
- **rdf:Property** – the class of properties.

RDFS Propertie

Properties are instances of the class rdf:Property and describe a relation between subject resources and object resources. When used as such a property is a predicate.

- **rdfs:domain** of an rdf:Property declares the class of the *subject* in a triple whose predicate is that property.
- **rdfs:range** of an rdf:Property declares the class or datatype of the *object* in a triple whose predicate is that property.

For example, the following declarations are used to express that the property ex:employer relates a subject, which is of type foaf:Person, to an object, which is of type foaf:Organization:

ex:employer rdfs:domain foaf:Person
ex:employer rdfs:range foaf:Organization

Given the previous two declarations, from the triple:

ex:John ex:employer ex:CompanyX

can be inferred (resp. follows) that ex:John is a foaf:Person, and ex:CompanyX is a foaf:Organization.

- **rdf:type** is a property used to state that a resource is an instance of a class. A commonly accepted qname for this property is "a".
- **rdfs:subClassOf** allows declaration of hierarchies of classes.

For example, the following declares that 'Every Person is an Agent':

foaf:Person rdfs:subClassOf foaf:Agent

Hierarchies of classes support inheritance of a property domain and range (see definitions in next section) from a class to its subclasses.

- **rdfs:subPropertyOf** is an instance of rdf:Property that is used to state that all resources related by one property are also related by another.
- **rdfs:label** is an instance of rdf:Property that may be used to provide a human-readable version of a resource's name.
- **rdfs:comment** is an instance of rdf:Property that may be used to provide a human-readable description of a resource.

RDF Schema

- RDF is a data model that provides a way to express simple **statements** about **resources**, using named **properties** and **values**.
- The **RDF Vocabulary Description Language 1.0 (or RDF Schema or RDFS)** is a language that can be used to define the **vocabulary** (i.e., the **terms)** to be used in an RDF graph.
- The RDF Vocabulary Description Language 1.0 is used to indicate that we are describing specific **kinds** or **classes** of resources, and will use specific **properties** in describing those resources.
- The RDF Vocabulary Description Language 1.0 is an **ontology** definition language (a **simple** one, compared with other languages such as OWL; **we can only define taxonomies and do some basic inference about them**).
- The RDF Vocabulary Description Language is like a **schema definition language** in the relational or object-oriented data models (hence the alternative name RDF Schema – we will use this name and its shorthand RDFS mostly!).

RDFS Conti...

- **The RDF Schema concepts are themselves provided in the form of an RDF vocabulary**; that is, as a specialized set of predefined RDF resources with their own special meanings.

- The resources in the RDF Schema vocabulary have URIrefs with the prefix `http://www.w3.org/2000/01/rdf-schema#` (associated with the QName prefix `rdfs:`).

- Vocabulary descriptions (schemas, ontologies) written in the RDF Schema language are **legal RDF graphs**. In other words, we use **RDF to represent RDFS information**.

Classes

- A basic step in any kind of description process is identifying the various kinds of things to be described. RDF Schema refers to these “kinds of things” as classes.
- A class in RDF Schema corresponds to the generic concept of a type or category, somewhat like the notion of a class in object-oriented programming languages such as Java or object-oriented data models.

Defining Classes

- Suppose an organization `example.org` wants to use RDF Schema to provide information about **motor vehicles, vans and trucks.**
- To define classes that represent these categories of vehicles, we write the following statements (triples):

```
ex:MotorVehicle rdf:type rdfs:Class .

ex:Van rdf:type rdfs:Class .
ex:Truck rdf:type rdfs:Class .
```

- **In RDFS, a class `C` is defined by a triple of the form**

```
C rdf:type rdfs:Class .
```

using the predefined class `rdfs:Class` and the predefined property `rdf:type`.

Defining Instances

- Now suppose `example.org` wants to define an individual car (e.g., the company car) and say that it is a motor vehicle.

- This can be done with the following RDF statement:

```
exthings:companyCar rdf:type ex:MotorVehicle .
```

rdf:type

- The predefined property `rdf:type` is used as a predicate in a statement

  `I rdf:type C`

  to declare that **individual `I` is an instance of class `C`.**

- In statements of the form (see previous slides

  `C rdf:type rdfs:Class .`

  `rdf:type` is used to declare that **class `C` (viewed as an individual object) is an instance of the predefined class `rdfs:Class`.**

Defining Classes Conti...

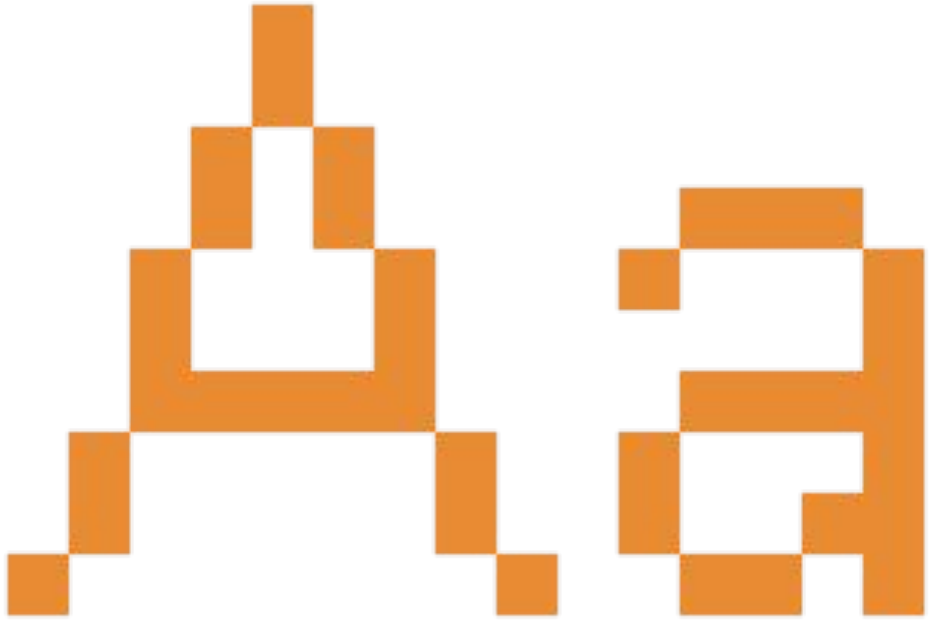
- **Defining a class explicitly is optional**; if we write the triple

```
I rdf:type C
```

then `C` is **inferred** to be a class (an instance of `rdfs:Class`) in RDFS.

Notation

Class names will be written with an initial uppercase letter, while property and instance names are written with an initial lowercase letter.



Defining Subclasses

- Now suppose `example.org` wants to define that vans and trucks are **specialized kinds** of motor vehicle.
- This can be done with the following RDF statements:

```
ex:Van    rdfs:subClassOf    ex:MotorVehicle .
ex:Truck  rdfs:subClassOf    ex:MotorVehicle .
```

- The predefined property `rdfs:subclassOf` is used as a predicate in a statement to declare that **a class is a specialization of another more general class.**
- A class can be a **specialization of multiple superclasses** (e.g., the graph defined by the `rdfs:subclassOf` property is a **directed graph** not a tree).

Classes and Instances

- The **meaning** of the predefined property `rdfs:subClassOf` in a statement of the form

  `C1 rdfs:subClassOf C2`

  is that any instance of class `C1` is also an instance of class `C2`.

- **Example:** If we have the statements

  ```
  ex:Van   rdfs:subClassOf   ex:MotorVehicle .
  exthings:myCar rdf:type ex:Van .
  ```

  then RDFS allows us to **infer** the statement

  ```
  exthings:myCar rdf:type ex:MotorVehicle .
  ```

Properties of `rdfs:subClassOf`

- The `rdfs:subClassOf` property is **reflexive** and **transitive**.

- **Examples:**
  - If we have a class `ex:MotorVehicle` then RDFS allows us to **infer** the statement

    ```
    ex:MotorVehicle rdfs:subClassOf ex:MotorVehicle .
    ```

  - If we have the statements

    ```
    ex:Van rdfs:subClassOf ex:MotorVehicle .
    ex:MiniVan rdfs:subClassOf ex:Van .
    ```

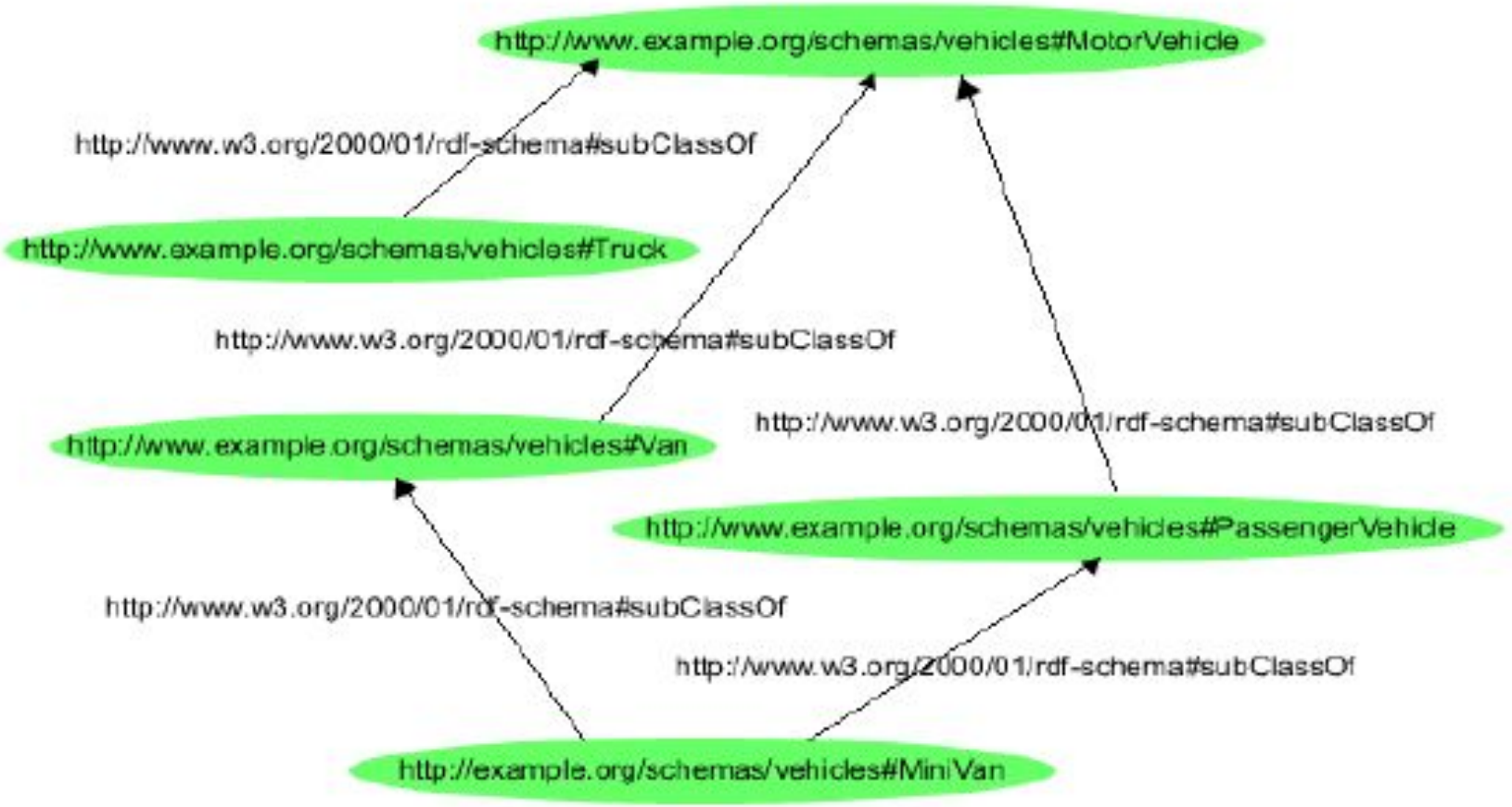
    then RDFS allows us to **infer** the statement

    ```
    ex:MiniVan rdfs:subClassOf ex:MotorVehicle .
    ```

RDF Schema Predefined Classes

- The group of resources that are RDF Schema classes **is itself a class** called `rdfs:Class`. **All classes are instances of this class.**
- In the literature, classes such as `rdfs:Class` that have other classes as instances are called **meta-classes**.
- All things described by RDF are called **resources**, and are instances of the class `rdfs:Resource`.
- `rdfs:Resource` **is the class of everything. All other classes are subclasses of this class.** For example, `rdfs:Class` is a subclass of `rdfs:Resource`.

Class and Instance Information as a Graph



The Graph in Triple Notation

```
ex:MotorVehicle rdf:type rdfs:Class .
ex:PassengerVehicle rdf:type rdfs:Class .
ex:Van rdf:type rdfs:Class .
ex:Truck rdf:type rdfs:Class .
ex:MiniVan rdf:type rdfs:Class .

ex:PassengerVehicle rdfs:subClassOf ex:MotorVehicle .
ex:Van rdfs:subClassOf ex:MotorVehicle .
ex:Truck rdfs:subClassOf ex:MotorVehicle .

ex:MiniVan rdfs:subClassOf ex:Van .
ex:MiniVan rdfs:subClassOf ex:PassengerVehicle .
```

Properties

In addition to defining the specific classes of things they want to describe; user communities also need to be able to define specific properties that characterize those classes of things (such as *author* to describe a book).

Defining Properties

- **A property can be defined by stating that it is an instance of the predefined class** `rdf:Property`.

- Example:

```
ex:author rdf:type rdf:Property .
```

- Then, property `ex:author` can be used as a **predicate in an RDF triple** such as the following:

```
ex:john ex:author ex:book123 .
```

- **Defining a property explicitly is optional**; if we write the RDF triple

```
S P O .
```

then `P` is **inferred** to be a property by RDFS.

Properties Conti...

- **Properties are resources too** (this makes RDF and RDFS different than many other KR formalisms).
- **Therefore, properties can appear as subjects or objects of triples.**
- Example (provenance):

```
ex:author prov:definedBy ke:john
ke:john prov:defined ex:author
```

Properties Conti...

- In RDFS property definitions are **independent** of class definitions. In other words, a property definition can be made without any reference to a class.
- Optionally, properties can be declared to apply to certain instances of classes by defining their **domain and range**.

Domain and Range

- RDFS provides vocabulary for describing **how properties and classes are intended to be used together** in RDF data.
- The `rdfs:domain` predicate can be used to indicate that a particular property applies to instances of a designated class (i.e., it defines the **domain** of the property).
- The `rdfs:range` predicate is used to indicate that the values of a particular property are instances of a designated class (i.e., it defines the **range** of the property).

EXAMPLE

```
ex:Book rdf:type rdfs:Class .
ex:Person rdf:type rdfs:Class .

ex:author rdf:type rdf:Property .

ex:author rdfs:domain ex:Book .
ex:author rdfs:range ex:Person .
```

Domain and Range Conti...

- For a property, we can have **zero, one, or more than one** domain or range statements.

- **No domain or no range statement**: If no range statement has been made for property `P`, then **nothing has been said about the values** of this property. Similarly for no domain statement.

- **Example:** If we have only the triple

```
exstaff:frank ex:hasMother exstaff:frances .
```

then nothing can be inferred from it regarding resources `exstaff:frank` and `exstaff:frances`.

- **One domain statement:** If we have

```
P rdfs:domain D .
```

then we can **infer** that when `P` is applied to some resource, this resource is an instance of class `D`.

- **One range statement:** If we have

```
P rdfs:range R .
```

then we can **infer** that when `P` is applied to some resource, the value of `P` is an instance of class `R`.

Examples –

[Domain – Subject
Range – Object]
S Property O

- If we have

```
ex:hasMother rdfs:domain ex:Person .
exstaff:frank ex:hasMother exstaff:frances .
```

then we can infer:

```
exstaff:frank rdf:type ex:Person.
```

- If we have

```
ex:hasMother rdfs:range ex:Person .
exstaff:frank ex:hasMother exstaff:frances .
```

then we can infer

```
exstaff:frances rdf:type ex:Person.
```

Domain and Range Conti...

- **Two domain or range statements:** If we have

```
P rdfs:range C1 .
P rdfs:range C2 .
```

then we can **infer** that the values of property `P` are instances of both `C1` and `C2`. Similarly, for two domain statements.

- **Example:** If we have

```
ex:hasMother rdfs:range ex:Female .
ex:hasMother rdfs:range ex:Person .
exstaff:frank ex:hasMother exstaff:frances .
```

then we can infer that `exstaff:frances` is an instance of both `ex:Female` and `ex:Person`.

EXAMPLE

```
ex:Human rdf:type rdfs:Class .
ex:hasParent rdf:type rdf:Property .
ex:hasParent rdfs:domain ex:Human .
ex:hasParent rdfs:range ex:Human .

ex:Tiger rdf:type rdfs:Class .
ex:hasParent rdfs:domain ex:Tiger .
ex:hasParent rdfs:range ex:Tiger .

ex:tina ex:hasParent ex:john .
```

Datatypes for Range

- The `rdfs:range` property can also be used to indicate that the value of a property is given by a **typed literal.**

- **Example:**

```
ex:age rdf:type rdf:Property .
ex:age rdfs:range xsd:integer .
```

- Optionally, we can also assert that `xsd:integer` is a **datatype** as follows:

```
xsd:integer rdf:type rdfs:Datatype .
```

Specialized Properties

- RDF Schema provides a way to **specialize properties** (similarly with classes). This specialization relationship between two properties is described using the predefined property `rdfs:subPropertyOf`.

- **Example:**

```
ex:driver rdf:type rdf:Property .
ex:primaryDriver rdf:type rdf:Property .
ex:primaryDriver rdfs:subPropertyOf ex:driver .
```

Specializing Properties Conti...

- If resources `S` and `O` are connected by the property `P1` and `P1` is a subproperty of property `P2`, then RDFS allows us to **infer** that `S` and `O` are also connected by the property `P2`.

- **Example:** If we have the statements

```
ex:john ex:primaryDriver ex:companyCar
ex:primaryDriver rdfs:subPropertyOf ex:driver .
```

then we can infer

```
ex:john ex:driver ex:companyCar .
```

Specializing Properties Conti...

- `rdfs:subPropertyOf` is **reflexive** and **transitive**.

- **Examples**:
  - If we have the property `ex:driver` then RDFS allows to infer the triple

    ```
    ex:driver rdfs:subPropertyOf ex:driver .
    ```

  - If we have the triples

    ```
    ex:primaryDriver rdfs:subPropertyOf ex:driver .
    ex:driver rdfs:subPropertyOf ex:isResponsibleFor .
    ```

    then RDFS allows us to **infer** the triple

    ```
    ex:primaryDriver rdfs:subPropertyOf ex:isResponsibleFor .
    ```

Some Utility Properties

- rdfs:label
- rdfs:comment
- rdfs:seeAlso
- rdfs:isDefinedBy

The Property rdfs:label

- `rdfs:label` is an instance of `rdf:Property` that may be used to provide a **human-readable version of a resource's name**.
- The `rdfs:domain` of `rdfs:label` is `rdfs:Resource`. The `rdfs:range` of `rdfs:label` is `rdfs:Literal`.
- Multilingual labels are supported using the **language tagging** facility of RDF literals.

The Property rdfs:comment

- `rdfs:comment` is an instance of `rdf:Property` that may be used to provide a **human-readable description of a resource.**
- The `rdfs:domain` of rdfs:comment is `rdfs:Resource`. The `rdfs:range` of `rdfs:comment` is `rdfs:Literal`.
- Multilingual documentation is supported through use of the **language tagging** facility of RDF literals.

The Property rdfs:seeAlso

- `rdfs:seeAlso` is an instance of `rdf:Property` that is used to indicate a resource that might provide additional information about the subject resource.

- A triple of the form `S rdfs:seeAlso O` states that the resource `O` may provide additional information about `S`. It may be possible to retrieve representations of `O` from the Web, but this is not required. When such representations may be retrieved, no constraints are placed on the format of those representations.

- The `rdfs:domain` of `rdfs:seeAlso` is `rdfs:Resource`. The `rdfs:range` of `rdfs:seeAlso` is `rdfs:Resource`.

The Property rdfs:isDefinedBy

- `rdfs:isDefinedBy` is an instance of `rdf:Property` that is used to indicate a resource defining the subject resource. This property may be used to indicate an RDF vocabulary in which a resource is described.

- A triple of the form `S rdfs:isDefinedBy O` states that the resource `O` defines `S`. It may be possible to retrieve representations of `O` from the Web, but this is not required. When such representations may be retrieved, no constraints are placed on the format of those representations. `rdfs:isDefinedBy` is a subproperty of `rdfs:seeAlso`.

- The `rdfs:domain` of rdfs:isDefinedBy is `rdfs:Resource`. The `rdfs:range` of `rdfs:isDefinedBy` is `rdfs:Resource`.

https://www.youtube.com/watch?v=PWjzAq3_CmE